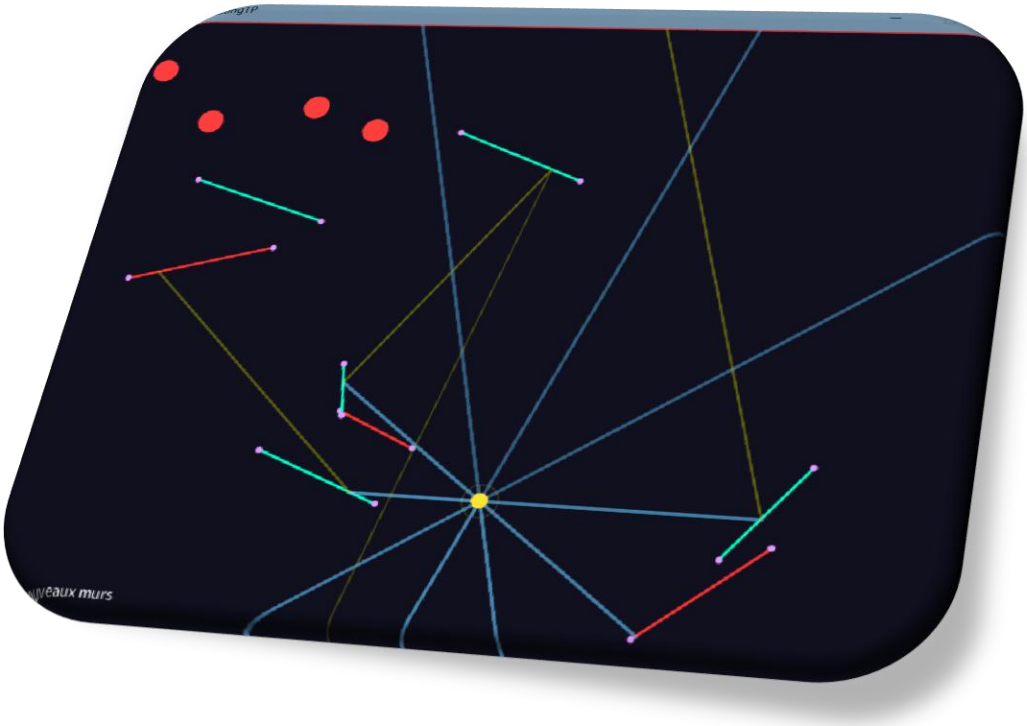
UQAC
Université du Québec
à Chicoutimi

RAPPORT DE TRAVAIL PRATIQUE

#1 Processing (casting et intersection de plan)



PAR

Mattéo BERTAULD-BECOURT

Mathis PLATEL-LENTZ

Oscar SIMON

Ilyes ZAIDI

March 23, 2026

Sommaire

Sommaire 2
Ressources utilisées 2
Explication de la solution 3
Architecture 3
Explication rapide 4
Analyse Mathématique : 5
Partie 1 : L'intersection segment-segment 5
Partie 2 : La réflexion vectorielle (rebond) 7
Discussion 9
Contributions des membres de l'équipe 9
Dépasse les attentes 10

Ressources utilisées

Téléchargent des exemples de la communauté fournis par le logiciel processing (https://processing.org/download)

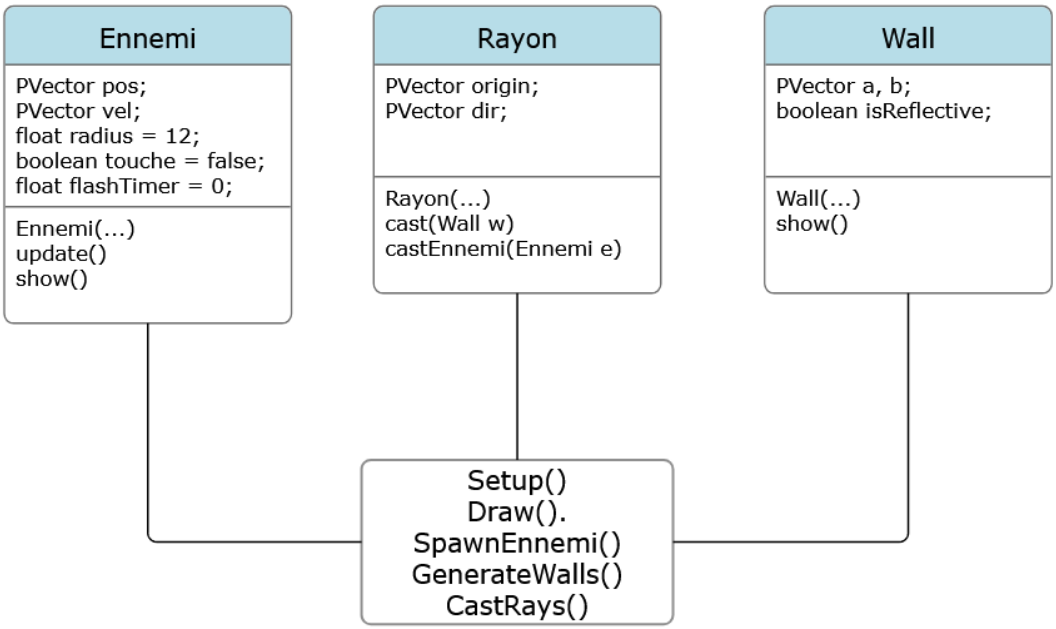
Utilisation de la documentation en ligne de processing pour comprendre le fonctionnement des méthodes (PVector, méthodes dot(), sub(), normalize())

Visionnage de tutoriels sur le *2D Raycasting* pour la structure de la boucle de rendu et la projection de rayon.
https://www.youtube.com/watch?v=TOEi6T2mtHo&pp=ygUZcHJvY2Vzc2luZyAyRCBSYXljYXN0aW5nIA%3D%3D

Utilisation de l'IA Gemini, principalement pour les détails visuels de la solution (effet de lumière, couleur, taille et émissivité variable des rayons, etc…)

Explication de la solution

Architecture



Le projet adopte une structure Orientée Objet pour séparer la logique physique du rendu visuel.

Les Classes

- Classe Rayon : Le cœur du moteur. Elle gère la projection dans l'espace, le calcul mathématique des intersections
- Classe Wall : Définit les segments d'obstacles via deux vecteurs (A et B). Elle porte l'attribut isReflective qui détermine si le mur est bloque ou renvoie la lumière.
- Classe Ennemi : Gère les entités mobiles

Organisation des Fonctions

Le fichier principal gère toutes les classes via quatre fonctions clés :

- Draw : Dessine la scène à chaque frame (60 FPS).
- Cast ray : La fonction logique qui génère les rayons initiaux et gère la cascade de rebonds.
- Generate wall : Initialise les bordures de l'écran et crée les obstacles aléatoires (regénérables avec la touche 'R').
- Spawn ennemi : Fait apparaître une nouvelle cible chaque seconde.

Explication rapide

La scène est délimitée par quatre murs de bordure permanents qui confinent les projections aux limites de la fenêtre (x=0,y=0,width,height), évitant ainsi tout calcul infini.
Quatre murs sont générés aléatoirement, que l'utilisateur peut régénérer à tout moment via la touche **'R'**.

Un ennemi simple spawn chaque seconde, celui-ci se déplace dans la scène et disparait si touché par un rayon

Le moteur simule la propagation de la lumière en projetant, à chaque image, un faisceau de n rayons répartis à 360° depuis la position du curseur.

Pour chaque rayon, le système exécute une boucle récursive

1. Le rayon analyse les intersections avec les murs et les ennemis ;
2. Si un adversaire se trouve sur la trajectoire, il est instantanément éliminé.
3. Analyse **du type mur touché :** Le système identifie la nature du premier mur touché pour décider de la suite de la propagation.
    - **Si le mur est réfléchissant (bleu),** un "rayon enfant" est généré au point d'impact avec un angle inversé, perdant en épaisseur et en intensité lumineuse à chaque rebond.
    - **Si le mur est bloquant (rouge),** le calcul s'interrompt et une petite ellipse est générée pour marquer visuellement le point de collision final.

Analyse Mathématique :

Partie 1 : L'intersection segment-segment

```
class Rayon {
  PVector origin;
  PVector dir;
  ...

PVector cast(Wall wall) {
    float x1 = wall.a.x;
    float y1 = wall.a.y;
    float x2 = wall.b.x;
    float y2 = wall.b.y;

    float x3 = origin.x;
    float y3 = origin.y;
    float x4 = origin.x + dir.x;
    float y4 = origin.y + dir.y;

    float denom = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4);

    // Rayon parallèle au mur
    if (denom == 0) return null;

    float t = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / denom;
    float u = -((x1 - x2) * (y1 - y3) - (y1 - y2) * (x1 - x3)) / denom;

    // Intersection valide : t entre 0 et 1 (sur le mur), u > 0 (dans la direction du rayon)
    if (t > 0 && t < 1 && u > 0) {
      float px = x1 + t * (x2 - x1);
      float py = y1 + t * (y2 - y1);
      return new PVector(px, py);
    }

    return null;
  }
}
```

1. Modélisation du problème

Tout point P situé sur un segment ou une droite peut être défini par l'équation vectorielle :

$$P=O+uD$$

- **O (Origine) :** Le point de départ.
- **D (Direction) :** Le vecteur directeur.
- **u (Paramètre) :** Un scalaire représentant la distance parcourue le long du vecteur.

Dans le code, nous récupérons les coordonnées des deux entités :

- **Le Mur :** Défini par les points (x1,y1) et (x2,y2).
- **Le Rayon :** Défini par son origine (x3,y3) et un point projeté dans sa direction (x4,y4).

Il nous faut vérifier s'il existe un point commun appartenant aux deux segments

2. Le Système d'Équations et le Déterminant

L'intersection existe si le point du mur est identique au point du rayon (Pmur = Prayon). Pour résoudre ce système, on calcule d'abord le **déterminant** (denom) :

$$denom=(x1-x2)(y3-y4)-(y1-y2)(x3-x4)$$

Note : Si denom == 0, les droites sont parallèles ou confondues. Il n'y a aucune intersection possible, le moteur ignore donc le calcul.

3. Résolution des Coefficients t et u

Si le dénominateur est valide, nous calculons les deux coefficients de proportionnalité qui nous indiquent **où** se trouve l'impact potentiel :

- **t (Position sur le mur) :**
  $$t = ((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4)) / denom$$
- **u (Profondeur du rayon) :**
  $$u = -((x1 - x2) * (y1 - y3) - (y1 - y2) * (x1 - x3)) / denom$$

4. La Condition de Collision

Une collision n'est enregistrée que si elle respecte les contraintes physiques de la scène :

- **0<t<1 :** L'impact se situe bien entre les deux extrémités du mur (il ne passe pas "à côté").
- **u>0 :** Le mur se trouve devant la source du rayon (il n'est pas "derrière").

5. Calcul des Coordonnées Finales

Si les conditions sont remplies, on injecte le coefficient t dans l'équation du mur pour obtenir les coordonnées cartésiennes exactes (px,py) du point de contact :

$$px=x1+t(x2-x1)$$
$$py=y1+t(y2-y1)$$

Partie 2 : La réflexion vectorielle (rebond)

```
RaycastingTP.pde

if (closestWall.isReflective) {
  float dx = closestWall.b.x - closestWall.a.x;
  float dy = closestWall.b.y - closestWall.a.y;
  PVector normal = new PVector(-dy, dx).normalize();
  float dot = rayDir.dot(normal);

  rayDir = PVector.sub(rayDir, PVector.mult(normal, 2 * dot));
  rayOrigin = closestPoint.copy();
}
```

Lorsqu'un rayon frappe un mur réfléchissant (bleu), nous voulons qu'il reparte avec un angle de sortie identique à son angle d'entrée, comme au billard.

1. Calcul du vecteur directeur du mur

On commence par définir le vecteur directeur du mur à partir de ses points d'extrémité

float dx = closestWall.b.x - closestWall.a.x;
float dy = closestWall.b.y - closestWall.a.y;

Ce vecteur v(dx,dy) représente l'orientation de la surface d'impact.

2. Construction de la Normale

En géométrie 2D, pour obtenir un vecteur perpendiculaire, on inverse les composantes dx et dy et on change l'un des signes :

PVector normal = new PVector(-dy, dx).normalize();

- **Vecteur :** (-dy, dx) est mathématiquement orthogonal à (dx, dy).
- **Normalisation :** La méthode .normalize() ajuste la longueur du vecteur à **1**.

3. Le Produit Scalaire (dot)

On calcule ensuite la projection du vecteur incident I

```
float dot = rayDir.dot(normal);
```

3. Calcule de la nouvelle direction

Pour obtenir la nouvelle direction R, on applique la formule suivante :

$$R = I - 2(I*n)*n$$

Réalisé par cette ligne de code

```
rayDir = PVector.sub(rayDir, PVector.mult(normal, 2 * dot));
```

Pourquoi ça fonctionne ?

- (I*n)*n est la composante verticale du rayon.
- En la soustrayant **une fois**, on annule la vitesse d'entrée (le rayon "glisse" alors parallèlement au mur).
- En la soustrayant **une deuxième fois**, on crée une vitesse de sortie opposée.
- La composante horizontale (parallèle au mur) reste inchangée, créant un rebond parfait.

4. Mise à jour de l'Origine

On met à jour le nouveau rayon :

```
rayOrigin = closestPoint.copy();
```

Discussion

Le projet accomplit l'ensemble des **fonctionnalités demandées** : les rayons sont projetés depuis le curseur, des murs sont générés aléatoirement dans la scène, et chaque rayon s'arrête correctement à la première intersection rencontrée.

La partie la plus complexe du projet a été l'implémentation de la réflexion des rayons. Trouver et comprendre la formule vectorielle $\mathbf{R = I - 2(I \cdot n)n}$ a demandé des recherches et plusieurs essais, notamment pour calculer correctement la normale au mur et l'appliquer à la direction du rayon.

Concernant les **limitations actuelles**, le nombre de rayons est fixe et ne peut pas être ajusté durant la partie ce qui ralenti le testing du projet. Les murs générés aléatoirement peuvent parfois se superposer ou être mal placés, affectant la lisibilité de la carte. Les ennemis se déplacent de façon linéaire. Enfin, le projet est en 2D. Pour pouvoir améliorer le TP on pourrait : ajouter d'un score, de murs mobiles et une évolution en 3D.

Contributions des membres de l'équipe

Le projet a été segmenté en trois pôles majeurs pour une répartition équitable des tâches :

- Mathis **sur la Gestion des Murs :** Développement de la classe Wall et programmation du système de génération des murs aléatoire (touche **'R'**).

- Mattéo & Oscar **sur la Physique des Rayons:** création des rayons et gestion du rebond sur les murs.

- Ilyes **sur le Système d'Ennemis :** Création de la classe Ennemi, apparition et détection de leur élimination.

Dépasse les attentes

Système de rebond (réflexion) : Les murs bleus renvoient le rayon. À chaque rebond, le rayon perd en épaisseur, en émissivité de lumière et change de couleur pour distinguer les rayons d'origine des rayons réfléchis.

Système d'ennemis : Des ennemis (cercles rouges) apparaissent et se déplacent dans la scène. Un rayon qui les touche les élimine, ce qui donne un aspect jeu au projet.
Polish visuel : Le code couleur (rouge = bloquant, bleu = réfléchissant) et le point d'impact marqué visuellement rendent la scène plus lisible.

Régénération de la carte : Les murs sont générés aléatoirement et peuvent être relancés avec la touche 'R' sans redémarrer le programme.